ML Architecture — Presenter Guide

English, topic-led lecture for a software engineering master's class. This guide matches the revised slide numbering. It supersedes the earlier incident-led speaking sequence. Slide count supports explanation; no fixed one-hour total is imposed. Coordinate actual speaking time with the group before delivery.

Shared example and boundaries

Bass Chapter 14 supplies the bank customer-churn setting. Monthly scoring, the next-30-day target, Ana/C042, login counts, the illustrative 0.70 score, M1/M2, the CRM migration and B017/B017-R1 are teaching choices. No actual customer outcome, repaired score, retention benefit, or executed tool run is reported.

The missing CRM feature is a data-quality fault, not automatically concept drift. The complete failed B017 batch is held before publication. A separately validated repaired execution B017-R1 can retain M1. Training and production promotion are separate decisions. Prediction and business action remain separate responsibilities.

Presenting and editing

Arrow Right/Space advances a diagram build, then the slide; Arrow Left reverses. Page Down/Page Up changes slides directly. G opens contents; Escape closes it. F toggles fullscreen. E toggles local text editing; Escape finishes editing; Ctrl/Cmd+S downloads the edited HTML. Browser edits are not synchronized to other laptops. The PDF and `?motion=static` show complete figures. Reduced motion also uses complete diagrams. No visible animation-playback controls are shown.

Section ownership

Section	Presenter
Foundations	Ricardo
Deployment and serving	Tomás
Monitoring and degradation	Simon
Design decisions	Luca
Architectural tactics	TBD
DevOps/MLOps	Matheus
Security and privacy	Simon + Matheus

Section	Presenter
Quality checklist	TBD
Practical tools	TBD

Unassigned responsibilities remain explicit; this document does not invent new speaker assignments. Detailed statistical derivations remain in Simon's personal study notes, rather than the main lecture.

1. ML Architecture

Section: Block 1 / Foundations. **Presenter:** All presenters.

From a trained model to a maintainable, observable, governed system.

- **Software engineering** for systems that depend on learned behavior.
- **One shared case:** bank customer-churn prediction.
- **Two blocks:** model-to-system foundations, then architecture and quality.
- **Five presenters:** Ricardo, Tomás, Simon, Luca, and Matheus.

Bank example: Bank churn supplies the shared case. Examples, records, schedules, and incidents are illustrative.

Teaching point: A useful model needs an engineered system around it.

Reading anchor: Bass, Ch. 14 · Kalinowski, Ch. 13–14.

Next: Two blocks, one shared bank case — All presenters.

2. Two blocks, one shared bank case

Section: Block 1 / Foundations. **Presenter:** All presenters.

Follow the group’s topic outline; use the case to connect each architectural decision.

- **Block 1:** lifecycle, serving, monitoring, and design decisions.
- **Block 2:** tactics, MLOps, security, privacy, and quality.
- **Concept first:** explain the mechanism, then apply it to churn.
- **Named handoffs:** each section identifies its presenter; unassigned topics remain TBD.

Bank example: The same source → feature → model → score → policy path returns throughout the lecture.

Teaching point: Each architectural decision connects predictive behavior to a system responsibility.

Reading anchor: Group discussion document · Agreed lecture redesign.

Next: The bank case: features, labels, and predictions — Ricardo.

3. The bank case: features, labels, and predictions

Section: Block 1 / Foundations. **Presenter:** Ricardo.

A few ML terms connect familiar application data to a learned risk estimate.

- **Feature:** a prepared input, such as logins during the previous seven days.
- **Label:** the observed churn outcome used for training and later evaluation.
- **Prediction:** an estimate produced before the customer's future outcome is known.
- **Training vs. inference:** learn from earlier examples; apply the approved model to current inputs.

Bank example: Ana / C042: logins fall from 7 to 1; the illustrative score is 0.70. Her next-30-day outcome is unknown.

Teaching point: Lower activity is evidence for a model input, not proof that a customer will leave.

Additional speaking context:

Ana's app logs are data. A consistent count of logins during the seven days ending at the scoring cutoff is a feature. A trained model turns prepared features into a risk estimate; in our invented example its output is 0.70. A label is the later observed churn outcome, defined consistently for learning and evaluation. Churn means ending the banking relationship, not simply opening the app less often.

Reading anchor: Bass, Ch. 14 · Simon's notes, §1.

Next: From a trained model to an ML system — Ricardo.

4. From a trained model to an ML system

Section: Block 1 / Foundations. **Presenter:** Ricardo.

Architecture connects learning, production data, operational controls, and business decisions.

- **Training** uses historical features and labels to build and evaluate candidates.
- **Production** validates current data and applies an approved, compatible model.
- **Stored scores** link predictions to customer, time, input evidence, and versions.
- **Business policy** determines the permitted action; it remains separate from the score.

Bank example: The bank computes a monthly batch of churn scores, then applies campaign eligibility and contact rules.

Teaching point: Keep the learning path and production path distinct, but traceable.

Additional speaking context:

Read the production path from left to right: source records, preparation and validation, scoring with M1, saved scores, then campaign policy. Below it, historical features and mature labels support candidate training and evaluation. Only a compatible approved model enters production. Later bank outcomes can be joined to saved scores to assess quality; campaign action history helps interpret what happened.

Reading anchor: Bass, Ch. 14 · Kalinowski, §13.2–13.3.

Next: Two lifecycles, one compatible system — Ricardo.

5. Two lifecycles, one compatible system

Section: Block 1 / Foundations. **Presenter:** Ricardo.

Code correctness and predictive usefulness depend on different feedback and release cycles.

- **Software changes** can be checked through code, contract, and integration tests.
- **Model development** also needs representative data, suitable labels, and evaluation.
- **Feedback differs:** execution checks may be immediate; churn labels arrive later.
- **Co-versioning** links changes across code, feature meaning, model, and configuration.

Bank example: A CRM connector can change today while M1 stays fixed; its changed inputs can still alter predictions.

Teaching point: A passing software test does not establish future predictive quality.

Additional speaking context:

Software engineers can change a connector or business rule, integrate the code, run tests and release it. Model development also needs suitable examples, feature definitions, learning and evaluation. Its feedback may arrive weeks after a prediction. The two activities depend on each other: changing input meaning can alter a prediction even when model bytes remain unchanged.

Reading anchor: Kalinowski, Ch. 13–14 · Bass, Ch. 11 · Simon’s notes, §1.

Next: Prediction and business action are separate — Ricardo.

6. Prediction and business action are separate

Section: Block 1 / Foundations. **Presenter:** Ricardo.

A model estimates risk. Business rules decide whether and how the bank may act.

- **Score:** estimated churn risk, not a guaranteed outcome.
- **Policy:** preferences, eligibility, contact limits, and campaign budget.
- **Action:** an offer, another response, or no contact at all.
- **Separate evaluation:** predictive quality and the benefit of an offer are different questions.

Bank example: Ana may have a high risk score and still receive no offer because contact preferences or policy prohibit it.

Teaching point: A policy change should not require retraining the prediction model.

Additional speaking context:

The score supports a retention process. The campaign must still consider contact preferences, eligibility, previous contacts and budget. A high-risk estimate does not mean the customer automatically receives an offer. No contact is also a possible policy outcome. Changing a contact-frequency rule should not require relearning the churn model.

Reading anchor: Bass, Ch. 14 · Teaching policy decomposition.

Next: Serving patterns: when is a prediction needed? — Tomás.

7. Serving patterns: when is a prediction needed?

Section: Block 1 / Deployment & serving. **Presenter:** Tomás.

Select the interaction pattern from the decision's timing and failure behavior.

- **Batch:** compute many predictions; consumers read stored results later.
- **Synchronous:** the caller waits for a result during its request.
- **Asynchronous:** an event producer and consumer are decoupled through messaging.
- **Edge:** a deployment location, independent of prediction timing.

Bank example: The churn campaign can use precomputed scores. An immediate fraud check motivates a different timing need.

Teaching point: Timing, location, and release strategy are separate choices.

Additional speaking context:

Batch processes many records and stores results for later consumption. In a synchronous interaction a caller waits for the response before continuing. In an asynchronous design, messages separate request or event production from processing and result delivery. The prediction may still need to arrive quickly even though the original caller does not wait on one direct request-response call.

Reading anchor: Kalinowski, §13.3–13.4; supporting Ch. 12.

Next: Batch serving: validate, score, publish — Tomás.

8. Batch serving: validate, score, publish

Section: Block 1 / Deployment & serving. **Presenter:** Tomás.

A complete validated batch must be available before its business deadline.

- **Snapshot:** define population, cutoff, and required source freshness.
- **Validate:** reject incompatible inputs before scoring.
- **Reconcile:** account for written, rejected, and withheld records.
- **Publish:** expose a validated batch under a defined consumer contract.

Bank example: Monthly churn scores are saved for the campaign. A real numerical deadline must be agreed with campaign owners.

Teaching point: Optimize the whole batch deadline, not only per-record prediction speed.

Additional speaking context:

Walk one normal batch: required sources arrive, records are checked, features are computed, the compatible M1 package is loaded, scores are stored and expected coverage is reconciled. The objective is a complete validated batch in time for its campaign. The team has not specified a real numerical deadline or availability target, so do not invent one.

Reading anchor: Bass, Ch. 14 · Kalinowski, §13.3.

Next: Synchronous serving: the caller waits — Tomás.

9. Synchronous serving: the caller waits

Section: Block 1 / Deployment & serving. **Presenter:** Tomás.

Prediction becomes part of the application’s response path.

- **Response time** includes lookup, preparation, inference, and network work.
- **Availability** now affects the waiting application’s request.

- **Failure behavior** needs explicit timeout, error, and fallback policies.
- **Consistency** still requires compatible feature meanings and model versions.

Bank example: An immediate fraud check can justify this pattern. The monthly churn campaign does not require it.

Teaching point: A fast model does not guarantee a fast end-to-end service.

Additional speaking context:

Batch processes many records and stores results for later consumption. In a synchronous interaction a caller waits for the response before continuing. In an asynchronous design, messages separate request or event production from processing and result delivery. The prediction may still need to arrive quickly even though the original caller does not wait on one direct request-response call.

Reading anchor: Kalinowski, §13.3–13.4 · Teaching comparison.

Next: Asynchronous serving: decouple the work — Tomás.

10. Asynchronous serving: decouple the work

Section: Block 1 / Deployment & serving. **Presenter:** Tomás.

Messages separate production, processing, and consumption of a prediction.

- **Buffering** absorbs bursts and supports independent processing.
- **Lag** and queue health become operational concerns.
- **Retries** require failure handling and idempotent processing.
- **Contracts** define message identity, timing, schema, and result interpretation.

Bank example: If a future campaign consumes prediction events, a repeated message must not cause duplicate customer contacts.

Teaching point: Decoupling adds messaging responsibilities; it does not remove timing requirements.

Additional speaking context:

Batch processes many records and stores results for later consumption. In a synchronous interaction a caller waits for the response before continuing. In an asynchronous design, messages separate request or event production from processing and result delivery. The prediction may still need to arrive quickly even though the original caller does not wait on one direct request-response call.

Reading anchor: Kalinowski, §13.3 · Bank application is hypothetical.

Next: Deployment location is a separate decision — Tomás.

11. Deployment location is a separate decision

Section: Block 1 / Deployment & serving. **Presenter:** Tomás.

Central, embedded, and edge deployments change operations and constraints.

- **Central:** simplify shared operation and governance.
- **Embedded:** deploy the model with its host application.
- **Edge:** account for constrained hardware and intermittent connectivity.
- **Choose independently:** location does not uniquely determine batch or request timing.

Bank example: Keep churn centrally governed. Device-local inference is a contrast, not an extra subsystem to implement here.

Teaching point: Architecture follows the operational context.

Additional speaking context:

A model can run centrally, embedded in an application, or near a device. Those locations do not by themselves specify batch, synchronous or asynchronous behavior. An edge deployment can respond locally; a central service can participate in a message-based architecture. Choose the location using resource, connectivity, ownership and operational requirements.

Reading anchor: Kalinowski, supporting Ch. 12; §13.3–13.4.

Next: Package the whole compatible set — Tomás.

12. Package the whole compatible set

Section: Block 1 / Deployment & serving. **Presenter:** Tomás.

A deployable release contains more than the learned model file.

- **Model artifact:** the learned parameters and required format.
- **Runtime:** dependencies, preparation code, and execution environment.
- **Contract:** feature meaning, types, missingness, and temporal cutoff.
- **Configuration:** scoring population and execution settings.

Bank example: The bank packages M1 with the exact feature interpretation needed to score the monthly customer snapshot.

Teaching point: A package that loads successfully still needs release and quality evidence.

Additional speaking context:

The learned model needs its dependencies, expected input schema, feature semantics and scoring configuration. Check that the intended runtime can load it and perform inference. A column with the same name but a different time window is not necessarily compatible. Containerization can standardize a runtime; it does not establish that the inputs represent the right business meaning.

Reading anchor: Bass, Ch. 14 · Kalinowski, §13.3–13.4.

Next: Shadow deployment: observe without acting — Tomás.

13. Shadow deployment: observe without acting

Section: Block 1 / Deployment & serving. **Presenter:** Tomás.

A candidate scores real inputs while the approved model remains the action source.

- **Run in parallel** against comparable production inputs.
- **Record identity:** distinguish M1 and M2 outputs explicitly.
- **Compare** operational behavior and score differences first.
- **Wait for labels** to compare future churn quality.

Bank example: Only the M1 record is eligible for campaign use; M2's shadow score cannot create a second offer.

Teaching point: Shadow checks observation, not the business effect of acting on the candidate.

Additional speaking context:

In shadow mode, M1 and candidate M2 may score the same batch, but only the approved action path can influence the campaign. Candidate outputs are comparison records. A canary lets a limited approved cohort use the new version while the remainder retains the old one. In both cases, customer eligibility must prevent duplicate offers.

Reading anchor: Group outline · Breck, infrastructure 6–7 · Teaching application.

Next: Canary deployment: limit exposure — Tomás.

14. Canary deployment: limit exposure

Section: Block 1 / Deployment & serving. **Presenter:** Tomás.

An approved limited cohort uses the candidate before broader rollout.

- **Restrict exposure** to a defined cohort and scope.
- **Observe** execution, compatibility, coverage, and policy behavior.

- **Define stop rules** and a rehearsed recovery path.
- **Separate evidence:** short operational success cannot prove 30-day predictive quality.

Bank example: The bank may authorize a small cohort to use M2 while retaining M1 for the remainder.

Teaching point: A canary controls risk; it is not automatically a valid business experiment.

Additional speaking context:

In shadow mode, M1 and candidate M2 may score the same batch, but only the approved action path can influence the campaign. Candidate outputs are comparison records. A canary lets a limited approved cohort use the new version while the remainder retains the old one. In both cases, customer eligibility must prevent duplicate offers.

Reading anchor: Breck, infrastructure test 6 · Bank teaching application.

Next: A/B testing: define the intervention — Tomás.

15. A/B testing: define the intervention

Section: Block 1 / Deployment & serving. **Presenter:** Tomás.

Assigned groups and a stated outcome make the comparison interpretable.

- **State what changes:** model, decision policy, or retention treatment.
- **Assign comparable groups** under an appropriate experiment design.
- **Define outcomes** and the observation window before interpreting results.
- **Avoid confounding:** changing model and offer together obscures their separate effects.

Bank example: Compare two offer policies separately from asking whether M2 ranks churn risk better than M1.

Teaching point: Prediction accuracy and the effect of contacting a customer are different targets.

Additional speaking context:

An experiment needs a clear question, appropriate group assignment and a measured outcome. State whether the alternative is the risk model, the campaign treatment, or another controlled change. If model selection and treatment change together, interpreting an observed retention difference becomes harder. Comparing people who received an offer with everyone else is not automatically a valid estimate of the offer's effect.

Reading anchor: Group outline · Simon’s notes, §3 · Teaching application.

Next: How the world changes over time — Simon.

16. How the world changes over time

Section: Block 1 / Monitoring & degradation. **Presenter:** Simon.

Sudden, gradual, incremental, and recurring changes are different time patterns—not different input features.

- **Sudden:** the old regime switches abruptly.
- **Gradual:** old and new regimes coexist during a transition.
- **Incremental / recurring:** intermediate change differs from a returning regime.
- **Blips:** isolated noise is not necessarily a lasting regime change.

Figure explanation: Figure requested by the presenter: https://www.researchgate.net/figure/Four-types-of-concept-drift-according-to-severity-and-speed-of-changes-and-noisy-blips_fig2_321627304 . The identical figure was retrieved intact from the publicly posted PyConf 2019 deck at https://pyconf.hydpy.org/PyConf19Slides/talks/The-Past-and-the-dubious-Concept-Drift.pdf , page 6, because direct ResearchGate image download returned HTTP 403. Source publication: B. Krawczyk and A. Cano, Online Ensemble Learning with Abstaining Classifiers for Drifting and Noisy Data Streams, Applied Soft Computing 68, 677–692, 2018. These temporal patterns do not replace the distinction between input drift, concept drift and invalid data. Blips are shown separately from lasting drift.

Bank example: Bank illustrations: a pricing change can be sudden; staggered adoption gradual; evolving habits incremental; seasonal activity recurring.

Teaching point: First ask how change unfolds over time. Then ask what changed: input data, outcome relationships, or data validity.

Reading anchor: Krawczyk & Cano (2018), Fig. 2 · Original figure; attribution and source link in guide.

Next: Monitoring: observe more than model accuracy — Simon.

17. Monitoring: observe more than model accuracy

Section: Block 1 / Monitoring & degradation. **Presenter:** Simon.

Inputs, system execution, predictions, and business actions need different evidence.

- **Inputs:** are the right records available and meaningful?
- **System:** did the pipeline complete correctly and on time?
- **Model:** are predictions useful on the current population?
- **Business:** are permitted actions and outcomes being tracked?

Bank example: A healthy churn job can still use a shifted population. A useful score can still lead to an inappropriate contact.

Teaching point: Observability connects evidence to investigation; dashboards alone do not decide the remedy.

Additional speaking context:

Ask whether the inputs are valid, whether training and production compute them consistently, whether the population changed, and whether the relationship to churn changed. A source-quality fault concerns absent, stale or incompatible records. Training-serving skew concerns inconsistent preparation. Input drift concerns the distribution of valid features. Concept drift concerns the relationship between valid inputs and the outcome.

Reading anchor: Simon's notes, §2–4 · Bass, Ch. 11 / §14.1.5.

Next: Monitoring architecture: from signal to response — Simon.

18. Monitoring architecture: from signal to response

Section: Block 1 / Monitoring & degradation. **Presenter:** Simon.

Collect linked evidence and route it to an owner who can investigate and act.

- **Signals:** input, execution, and predictive evidence arrive at different times.
- **Context:** link observations to run, feature, model, and cohort identity.
- **Routing:** an alert has a defined recipient and response policy.
- **Authority:** investigation and release approval remain separate.

Bank example: A CRM missingness alert links to B017 and reaches source / operations owners; it does not promote M2.

Teaching point: A useful monitoring design connects evidence, ownership, and a controlled response.

Additional speaking context:

Start with the three evidence sources. Input quality and run health can be assessed immediately; predictive quality requires outcomes linked back to the original prediction context. The evidence store represents linked logs, metrics and reports, not a required single product. Alert routing brings a defined condition to a named owner. That owner reviews the supporting evidence and selects a controlled response. Review authority does not bypass the separate release policy.

Reading anchor: Simon's notes, §2–4 · Bass, Ch. 11 / §14.1.5 · Teaching expansion of monitoring responsibilities.

Next: Data drift: a different customer mix — Simon.

19. Data drift: a different customer mix

Section: Block 1 / Monitoring & degradation. **Presenter:** Simon.

Compare the same input definition across a reference cohort and the current cohort.

- **What changes:** the distribution of valid model inputs.
- **Bank example:** a campaign attracts more low-activity customers.
- **Available now:** compare inputs without waiting for churn labels.
- **Limit:** a changed distribution alone does not prove lost accuracy.

Bank example: Both diagrams count 100 fictional customers using the same seven-day login bins. Only the customer mix changes.

Teaching point: Choose a meaningful reference, comparable cohorts, and a defined response.

Additional speaking context:

Read the bin labels first: 0, 1–2, 3–5, and 6+ app logins in the same seven-day window. Each bar is a count within a fictional 100-customer cohort; this is a categorical proportion comparison, not a measured bank distribution. The current cohort contains more low-activity customers. Input evidence alone can establish this change, without knowing whether those customers later churned. It does not establish whether model errors increased. Reference selection and cohort comparability matter as much as the comparison method.

Reading anchor: Simon's notes, §2–4 · Bass, Ch. 11 / §14.1.5 · Invented counts.

Next: Concept drift: a different relationship — Simon.

20. Concept drift: a different relationship

Section: Block 1 / Monitoring & degradation. **Presenter:** Simon.

The same inputs can have different outcomes while the deployed model keeps its earlier boundary.

- **What changes:** churn risk given the same valid input information.
- **Bank example:** new fees make active customers leave more often.
- **Evidence:** link predictions to mature, reliable churn outcomes.
- **Model status:** M1 stays fixed until a separately evaluated release changes it.

Bank example: The two panels use the same 15 fictional feature positions. After the hypothetical change, some previously staying customers churn.

Teaching point: A missing field is not concept drift; a statistical alarm is not a retraining approval.

Additional speaking context:

Explain both axes and the legend first: app logins and products used are inputs; circles and triangles are mature observed outcomes. The same 15 positions are used twice to isolate a changed relationship. The gray dashed M1 boundary stays fixed in both panels. The solid line illustrates the changed outcome relationship, not an automatically deployed replacement. These deliberately simplified examples are invented and do not claim measured bank performance.

Reading anchor: Simon's notes, §2–4 · Bass, Ch. 11 / §14.1.5 · Schematic relationships.

Next: Concept drift: how M1 starts making errors — Simon.

21. Concept drift: how M1 starts making errors

Section: Block 1 / Monitoring & degradation. **Presenter:** Simon.

A fixed model can miss churners because the world changed—not because its software stopped running.

- **Inputs stay fixed:** the two cohorts use identical feature positions.
- **M1 stays fixed:** its earlier decision boundary is unchanged.
- **Outcomes change:** a different relationship creates missed churners.
- **Evidence:** the example uses mature labels, not outcomes already known on scoring day.

Figure explanation: Read the shaded region between the old dashed model boundary and the new solid outcome relationship. Rings mark customers who stayed in the reference scenario but churn in the changed scenario. The two labeled customer examples make the conditional change explicit. This perfect-separation construction is intentionally simple; actual populations can overlap and be noisy. The solid line describes the changed teaching relationship, not a replacement model that has already been trained or released.

Bank example: C1 has 5 logins and 5 products; C2 has 11 logins and 1 product. The same inputs now accompany different observed churn outcomes.

Teaching point: Changed labels reveal the mismatch. Investigate causes, evaluate candidates, and use controlled release rather than automatic promotion.

Reading anchor: Simon's notes, §2–4 · Bass, Ch. 11 / §14.1.5 · Invented mature cohorts and deliberately simplified decision rule.

Next: Input checks now; quality checks later — Simon.

22. Input checks now; quality checks later

Section: Block 1 / Monitoring & degradation. **Presenter:** Simon.

The target window creates a delay between a prediction and complete outcome evidence.

- **At scoring time:** check freshness, missingness, execution, and coverage.
- **During the window:** the full churn outcome is still unknown.
- **After ingestion:** join labels to the original score and target window.
- **Evaluate by cohort:** precision, recall, and calibration answer different questions.

Bank example: Ana’s D0 score is saved with its context. Complete next-30-day evaluation must wait for that window and ingestion.

Teaching point: An absent label does not mean the customer stayed.

Additional speaking context:

At scoring time, check freshness, completeness, coverage, valid features, job completion and score distributions. Ana's 30-day outcome is still pending. After the defined window closes and labels arrive reliably, join outcomes to the original saved predictions. Record prediction time, target window and label definition so that next month's outcome is not attached to this month's score.

Reading anchor: Simon’s notes, §2–4 · Bass, Ch. 11 / §14.1.5 · Teaching 30-day window.

Next: Drift detection: methods and their purpose — Simon.

23. Drift detection: methods and their purpose

Section: Block 1 / Monitoring & degradation. **Presenter:** Simon.

Methods compare inputs; they do not directly measure business value or model correctness.

- **Reference:** name the population, period, and feature definition.
- **Thresholds:** calibrate for the application rather than copying a universal rule.
- **Segments:** inspect affected cohorts instead of relying only on an aggregate.
- **Action:** investigate drift alongside quality, change, and business evidence.

Bank example: Compare the same seven-day login feature before and after a campaign; investigate why the cohort mix changed.

Teaching point: Interpret the signal alongside data validity, sample size, and downstream quality.

Additional speaking context:

The notes compare activity groups changing from 60/30/10% to 30/40/30%. No churn labels are needed for that comparison. PSI summarizes changes in common bins, KL compares distributions directionally, and JS provides a symmetric comparison. They have different properties and scales. KS instead uses the largest

cumulative-distribution gap, with test assumptions that must fit the input type.

Reading anchor: Simon's notes, §2–4 · Bass, Ch. 11 / §14.1.5 · Simon's technical atlas.

Next: Data-quality faults and training-serving skew — Simon.

24. Data-quality faults and training-serving skew

Section: Block 1 / Monitoring & degradation. **Presenter:** Simon.

Invalid data and inconsistent preparation differ from a shifted valid population.

- **Data-quality fault:** missing, stale, corrupt, or incompatible records.
- **Skew:** training and production prepare the feature differently.
- **Inspect:** definitions, cutoffs, joins, source changes, and code versions.
- **Repair the cause:** retraining on defective inputs can preserve the defect.

Bank example: A missing engagement field after CRM migration is invalid input. A 30-day production count replacing seven days is skew.

Teaching point: Do not use the word "drift" as a diagnosis for every alarm.

Additional speaking context:

Ask whether the inputs are valid, whether training and production compute them consistently, whether the population changed, and whether the relationship to churn changed. A source-quality fault concerns absent, stale or incompatible records. Training-serving skew concerns inconsistent preparation. Input drift concerns the distribution of valid features. Concept drift concerns the relationship between valid inputs and the outcome.

Reading anchor: Simon's notes, §2–4 · Bass, Ch. 11 / §14.1.5.

Next: A publication gate contains a bad batch — Simon.

25. A publication gate contains a bad batch

Section: Block 1 / Monitoring & degradation. **Presenter:** Simon.

The CRM incident illustrates a concrete input fault and a controlled response.

- **Detect:** invalid records violate the agreed input contract.
- **Contain:** hold the entire B017 batch before publication.
- **Investigate:** relate cohort, source change, feature definition, and execution.

- **Keep M1:** a source fault does not itself justify a new learned model.

Bank example: Ana's engagement value is missing, not zero. No new score or completed customer contact is invented.

Teaching point: A gate blocks unsafe use; an alert sends evidence to someone who can investigate.

Additional speaking context:

A CRM migration makes engagement values unavailable for part of B017. M1 is unchanged. Missing values are not evidence of zero app activity; replacing them with zero would create a new unvalidated interpretation. The input gate holds the entire batch before publication, so no new B017 scores are released for campaign use.

Reading anchor: Simon's notes, §2–4 · Bass, Ch. 11 / §14.1.5 · Fictional B017 incident.

Next: Respond: investigate, contain, repair, verify — Simon.

26. Respond: investigate, contain, repair, verify

Section: Block 1 / Monitoring & degradation. **Presenter:** Simon.

A monitoring signal opens a decision process; it does not select or approve a remedy.

- **Diagnose** before choosing repair, rollback, fallback, or training.
- **Preserve evidence** and the identity of failed attempts.
- **Validate the complete path** after the chosen repair.
- **Request a model change** only when evidence supports one.

Bank example: Repair the CRM path, validate a new execution B017-R1, and retain M1 unless a separate model change is justified.

Teaching point: Handoff to Luca: which dependencies made this failure possible?

Additional speaking context:

Use B017 to locate the CRM export, feature definition, code and last healthy run. The notes' optional example has 170 missing values among 1,000 expected rows, compared with a 0.5% reference and an invented 2% hold threshold. The denominator includes the expected population; counting only received rows could conceal missing records. The gate holds publication; an assigned owner investigates the change.

Reading anchor: Simon's notes, §2–4 · Bass, Ch. 11 / §14.1.5.

Next: Hidden dependencies create technical debt — Luca.

27. Hidden dependencies create technical debt

Section: Block 1 / Design decisions. **Presenter:** Luca.

An innocent upstream change can alter ML behavior without changing the model.

- **Hidden coupling:** features depend on sources owned by other teams.
- **Pipeline complexity:** parallel preparation paths can diverge.
- **Versioning gaps:** a model artifact alone cannot reproduce a decision.
- **Context matters:** architecture decisions depend on requirements and constraints.

Bank example: CRM changes its engagement export for another purpose; both training and production can inherit the change.

Teaching point: Make data dependencies as reviewable as code dependencies.

Additional speaking context:

The phrase “CRM feeds the model” hides assumptions about identity, timestamps, feature meaning and ownership. Define engagement_7d as the count of logins during the seven days ending at the scoring cutoff. Include type, missingness policy, compatible preparation and the team responsible for source changes. The migration must preserve those meanings or initiate coordinated review.

Reading anchor: Kalinowski, Ch. 14 · Nazir, §§5–6.

Next: Make the feature contract explicit — Luca.

28. Make the feature contract explicit

Section: Block 1 / Design decisions. **Presenter:** Luca.

A contract records meaning and ownership as well as structural schema.

- **Semantics:** define what the feature actually measures.
- **Time:** specify window, cutoff, and joins consistently.
- **Validity:** define types, ranges, freshness, and missingness.
- **Ownership:** review upstream changes with affected consumers.

Bank example: engagement_7d means app logins during seven days ending at the scoring cutoff; missing must not silently become zero.

Teaching point: Schema checks alone cannot detect every change in meaning.

Additional speaking context:

The phrase “CRM feeds the model” hides assumptions about identity, timestamps, feature meaning and ownership. Define engagement_7d as the count of logins during the seven days ending at the scoring cutoff. Include type, missingness policy, compatible preparation and the team responsible for source changes. The migration must preserve those meanings or initiate coordinated review.

Reading anchor: Bass, Ch. 14 · Nazir, §§5–6 · Teaching contract.

Next: Separate responsibilities where change happens — Luca.

29. Separate responsibilities where change happens

Section: Block 1 / Design decisions. **Presenter:** Luca.

Clear interfaces let different concerns evolve without unnecessary relearning.

- **Single responsibility:** identify each module’s reason to change.
- **Interfaces:** specify data and behavior at each boundary.
- **Dependency injection:** provide implementations through explicit contracts.
- **Deployment:** logical modules do not automatically require separate services.

Bank example: A connector fix changes preparation; a contact-frequency rule changes campaign policy; a new M2 changes inference.

Teaching point: Separate concerns first; choose deployment units from the actual trade-offs.

Additional speaking context:

Ingestion obtains source records. Preparation checks and computes features. Scoring applies the approved learned rule. Storage preserves results and version links. Campaign policy decides appropriate action. A connector repair belongs near preparation; a contact-limit change belongs to campaign policy. Those different reasons for change should remain understandable and testable.

Reading anchor: Kalinowski, Ch. 14 · Nazir, §§5–6.

Next: Pipeline pattern: order the responsibilities — Luca.

30. Pipeline pattern: order the responsibilities

Section: Block 1 / Design decisions. **Presenter:** Luca.

Make preparation and validation steps explicit and testable.

- **Explicit stages** clarify transformations and dependencies.
- **Stage contracts** make intermediate data understandable.

- **Tests** cover local behavior and end-to-end compatibility.
- **Failure handling** prevents a partial result from masquerading as a complete batch.

Bank example: Validate CRM records before computing engagement_7d and invoking M1.

Teaching point: Use structure to expose the dependency, not to hide it in orchestration code.

Additional speaking context:

Pipeline divides processing into steps, placing required validation before scoring. Strategy provides a stable interface behind which a preparation or algorithm implementation can change. Observer notifies interested components of an event, such as a monitoring result. These solve different design problems. An interface-compatible replacement still needs behavioral validation, and a notification does not authorize promotion.

Reading anchor: Kalinowski, Ch. 14 · Nazir, §§5–6.

Next: Strategy pattern: replace one implementation — Luca.

31. Strategy pattern: replace one implementation

Section: Block 1 / Design decisions. **Presenter:** Luca.

A stable interface separates the client from a specific implementation choice.

- **Stable interface:** the client relies on a contract, not a specific class.
- **Alternative implementations** can be supplied through that interface.
- **Compatibility** still includes meaning, not only types.
- **Evaluation** and release approval still apply when behavior changes.

Bank example: Replace a feature-preparation implementation behind a stable interface; verify equivalent semantics before scoring.

Teaching point: Interchangeability in code is not equivalence in predictive behavior.

Additional speaking context:

Pipeline divides processing into steps, placing required validation before scoring. Strategy provides a stable interface behind which a preparation or algorithm implementation can change. Observer notifies interested components of an event, such as a monitoring result. These solve different design problems. An interface-compatible replacement still needs behavioral validation, and a notification does not authorize promotion.

Reading anchor: Kalinowski, Ch. 14 · Nazir, §§5–6.

Next: Observer pattern: notify, then investigate — Luca.

32. Observer pattern: notify, then investigate

Section: Block 1 / Design decisions. **Presenter:** Luca.

A change notification decouples the signal producer from interested responders.

- **Subject:** emits a meaningful change event.
- **Subscribers:** receive notifications without tightly coupling the producer.
- **Context:** include cohort, reference, and evidence links.
- **Authority:** receiving a notification does not grant release privileges.

Bank example: A missingness event notifies data and operations owners; it does not instruct the registry to promote M2.

Teaching point: A useful notification requests attention and preserves the decision boundary.

Additional speaking context:

Pipeline divides processing into steps, placing required validation before scoring. Strategy provides a stable interface behind which a preparation or algorithm implementation can change. Observer notifies interested components of an event, such as a monitoring result. These solve different design problems. An interface-compatible replacement still needs behavioral validation, and a notification does not authorize promotion.

Reading anchor: Kalinowski, Ch. 14 · Nazir, §§5–6.

Next: Version the system, not only the model — Luca.

33. Version the system, not only the model

Section: Block 1 / Design decisions. **Presenter:** Luca.

A prediction depends on a compatible set of data meaning, code, model, and configuration.

- **Data meaning:** record source and feature definitions.
- **Code:** identify preparation and scoring revisions.
- **Model:** identify artifact, runtime, and configuration.
- **Outputs:** connect the compatible set to score and decision records.

Bank example: M1 alone cannot explain Ana’s score if a CRM migration changed the meaning of its engagement input.

Teaching point: Co-versioning connects system and model lifecycles.

Additional speaking context:

The same M1 artifact can receive features computed differently or run with different configuration. A score record therefore needs links to the relevant source/feature definition, preparation code, dependencies, model, scoring configuration and time. These links let the team identify which compatible set actually produced a batch.

Reading anchor: Kalinowski, Ch. 13–14 · Bass, Ch. 11 / §14.1 · Simon's notes.

Next: Modules or services? Make the trade-off — Luca.

34. Modules or services? Make the trade-off

Section: Block 1 / Design decisions. **Presenter:** Luca.

Independent deployment is useful only when it justifies its operating cost.

- **Modules** can provide strong logical boundaries with simpler operation.
- **Services** can support independent change and scaling.
- **Distribution** adds failure modes and version coordination.
- **Evidence** and requirements should drive the choice.

Bank example: Separate preparation from policy logically; deploy them independently only when the bank's needs justify it.

Teaching point: There is no universal best ML architecture.

Additional speaking context:

Modules can give a clear separation of responsibilities with simpler operation. Services may support independent ownership, deployment and scaling, but they introduce communication failures, interface evolution, release coordination and distributed recovery. Evaluate those benefits against the actual bank team's needs and capacity.

Reading anchor: Kalinowski, Ch. 14 · Nazir, §§5–6.

Next: Architectural tactics connect concerns to controls — TBD.

35. Architectural tactics connect concerns to controls

Section: Block 2 / Architectural tactics. **Presenter:** TBD.

Use explicit mechanisms to address uncertainty, change, recovery, and auditability.

- **Separation** isolates data preparation, model execution, and business policy.
- **Observation** makes operational and predictive behavior reviewable.

- **Recovery** restores a compatible path rather than only one artifact.
- **Governance** links evidence, approval, access, and responsibility.

Bank example: A validation boundary contains a bad CRM batch; version links and approvals support controlled repair.

Teaching point: A tactic is credible when its mechanism, owner, and evidence are clear.

Additional speaking context:

Ingestion obtains source records. Preparation checks and computes features. Scoring applies the approved learned rule. Storage preserves results and version links. Campaign policy decides appropriate action. A connector repair belongs near preparation; a contact-limit change belongs to campaign policy. Those different reasons for change should remain understandable and testable.

Reading anchor: Bass, Ch. 14 · Bass, Ch. 9–11 · Group Block 2 outline.

Next: CI, CD, and CT need different evidence — Matheus.

36. CI, CD, and CT need different evidence

Section: Block 2 / DevOps & MLOps. **Presenter:** Matheus.

Integration, delivery, deployment, and training automate different responsibilities.

- **CI:** integrate changes and run code, contract, and path checks.
- **Delivery:** keep a compatible change ready for an approval decision.
- **Deployment:** automatically release changes passing the defined policy.
- **CT:** train and evaluate candidates; completion is not production approval.

Bank example: A connector repair can use CI/CD while retaining M1. Retraining is not required for every pipeline change.

Teaching point: Automation executes policy; it does not make acceptance criteria identical.

Additional speaking context:

Continuous integration checks combined code changes, including the connector and preparation path. Continuous delivery keeps validated changes ready for a release decision; continuous deployment automatically releases changes satisfying configured gates. Continuous training repeatedly prepares and evaluates model candidates from suitable data. These activities can be connected without being identical.

Reading anchor: Bass, §14.1 · Kalinowski, §16.3–16.5 (supporting).

Next: Candidate training ends before promotion — Matheus.

37. Candidate training ends before promotion

Section: Block 2 / DevOps & MLOps. **Presenter:** Matheus.

A registry stores candidate evidence; a separate release decision controls production use.

- **Reproducibility:** record data, code, configuration, and artifacts.
- **Evaluation:** compare candidate evidence against the baseline and relevant cohorts.
- **Registration:** preserve identity and lineage for review.
- **Promotion:** authorize a specific compatible set with recovery planned.

Bank example: M2 remains a candidate until reviewed. M1 remains the approved scoring artifact during the CRM repair.

Teaching point: The registry is part of governance, not the whole approval process.

Additional speaking context:

Read the illustrative release record by function: what changed, which versions belong together, which checks ran, who approved, and how to recover. Feature and code references identify preparation. M1 identifies the learned artifact. Configuration and execution time identify the scoring context. Check results and approval refer to that specific release, not to a floating “latest” model.

Reading anchor: Bass, §14.1 · Kalinowski, §16.3–16.5 (supporting).

Next: Repair the input path; retain M1 when appropriate — Matheus.

38. Repair the input path; retain M1 when appropriate

Section: Block 2 / DevOps & MLOps. **Presenter:** Matheus.

The CRM incident requires a verified source repair, not an assumed new model.

- **Repair** the source or preparation defect.
- **Version** the reviewed fix and compatible configuration.
- **Re-run** under a new execution identity and validate the complete path.
- **Publish** only after the configured evidence and approval requirements pass.

Bank example: B017 is the held attempt. B017-R1 links the reviewed fix, compatible M1, validation evidence, and publication approval.

Teaching point: A successful fix needs evidence at the consumer boundary, not only a passing connector test.

Additional speaking context:

B017 failed before publication. Review the connector or mapping correction, restore complete source inputs, recompute features and identify the new execution as B017-R1. Check row coverage, feature meaning, required fields, compatibility with M1, stored outputs and campaign consumption. The approved learned rule can remain M1 because the demonstrated fault was in the input path.

Reading anchor: Bass, §14.1 · Kalinowski, §16.3–16.5 (supporting) · Simon's notes, §4.

Next: Trace the prediction and the decision — Matheus.

39. Trace the prediction and the decision

Section: Block 2 / DevOps & MLOps. **Presenter:** Matheus.

Lineage connects input evidence, execution, approval, and downstream action.

- **Prediction identity** links customer, time, and target window.
- **Execution evidence** identifies compatible code, features, model, and config.
- **Decision evidence** records policy version and action.
- **Access controls** protect sensitive reconstruction material.

Bank example: An investigator can trace Ana's original score and the campaign's separate no-contact or contact decision.

Teaching point: A model version alone cannot reconstruct the full decision.

Additional speaking context:

Start with Ana's prediction identifier. Retrieve the feature definition and protected input snapshot or sufficient historical reference, M1, scoring configuration, cutoff, target window and campaign-policy version. The action record may say no contact. The later outcome reference remains pending until available. Reconstructing a score and reconstructing a policy action are related but different tasks.

Reading anchor: Bass, §14.1 · Kalinowski, §16.3–16.5 (supporting) · Simon's notes, §4–6.

Next: Recovery restores a compatible path — Matheus.

40. Recovery restores a compatible path

Section: Block 2 / DevOps & MLOps. **Presenter:** Matheus.

Rehearse restoration before release, and verify it at the consumer boundary.

- **Know the target:** identify the last compatible release or validated fallback.

- **Rehearse:** test the recovery procedure before relying on it.
- **Restore the set:** include data meaning, runtime, and configuration.
- **Verify:** inspect stored scores and actual consumer behavior.

Bank example: Rolling M1 back cannot repair a CRM field that is still missing; the input path must be restored.

Teaching point: Handoff to security: who may read, change, approve, or operate these paths?

Additional speaking context:

Identify what a rollback restores: preparation, configuration, model references, batch outputs and downstream handling may all matter. Reverting the unchanged M1 cannot restore the missing CRM field. A fallback is a previously defined alternative, such as pausing automation or a validated manual process; old scores are not automatically suitable simply because they are available.

Reading anchor: Breck, infrastructure tests 6–7 · Simon's notes, §4.

Next: Security threats depend on attacker access — Simon + Matheus.

41. Security threats depend on attacker access

Section: Block 2 / Security & privacy. **Presenter:** Simon + Matheus.

Locate the attacker's control before naming the ML attack mechanism.

- **Poisoning:** altered learning inputs influence the trained model.
- **Inference manipulation:** altered production inputs affect an existing model's output.
- **Privacy inference:** access can reveal sensitive membership or attributes.
- **Assumptions matter:** access, knowledge, and goals differ by threat.

Bank example: A compromised CRM writer could affect current inputs; those records create a poisoning path only if later used for training.

Teaching point: The accidental CRM migration remains a different scenario.

Additional speaking context:

Explicitly open a hypothetical branch: suppose a compromised writer alters engagement records. Manipulating records that later enter learning creates a poisoning opportunity. Manipulating features consumed by an already trained model concerns prediction-time inputs. Privacy inference concerns learning sensitive information through outputs or model access. These are different mechanisms with different prerequisites.

Reading anchor: Simon's notes, §5–7 · Bass, Ch. 9–10 / §14.2 · NIST AI 100-2e2025.

Next: Enforce access at the resource boundary — Simon + Matheus.

42. Enforce access at the resource boundary

Section: Block 2 / Security & privacy. **Presenter:** Simon + Matheus.

Authentication, authorization, validation, and audit solve different parts of the problem.

- **Authenticate:** establish caller identity.
- **Authorize:** check operation, resource, and scope.
- **Validate:** enforce the input contract independently.
- **Audit:** preserve useful access and change evidence.

Bank example: The scorer may read the approved M1 artifact; it must not gain permission to promote M2 merely by reaching the registry.

Teaching point: A trust boundary needs an enforced policy, not only a box on a diagram.

Additional speaking context:

The campaign presents a credential to read its approved score view. Authentication establishes the identity; authorization checks its requested operation on that resource. An allowed read returns only permitted fields. A write or unrestricted CRM read is denied without returning protected data. Record the decision and reason in restricted audit evidence without logging the secret credential.

Reading anchor: Simon’s notes, §5–7 · Bass, Ch. 9–10 / §14.2 · NIST SP 800-207, §3.

Next: Least privilege follows responsibilities — Simon + Matheus.

43. Least privilege follows responsibilities

Section: Block 2 / Security & privacy. **Presenter:** Simon + Matheus.

Separate source writes, candidate creation, release approval, and campaign use.

- **Scope identities** to the operations and resources they need.
- **Separate duties:** training does not imply release authority.
- **Protect credentials** and avoid secrets in code or broad logs.
- **Review changes:** connect access, source updates, and approvals.

Bank example: A campaign user reads an approved score view, not unrestricted training data or production deployment credentials.

Teaching point: This abbreviated teaching matrix is a starting point, not a complete bank permission design.

Additional speaking context:

Read the matrix by identity and operation. A feature job reads authorized source data and writes prepared features. A trainer creates candidates. A release reviewer can promote an approved compatible set. The scorer reads approved artifacts and writes scores; the campaign reads a restricted approved view. Write access does not silently include promotion authority.

Reading anchor: Simon's notes, §5–7 · Bass, Ch. 9–10 / §14.2 · Proposed permissions.

Next: Privacy: expose what each role needs — Simon + Matheus.

44. Privacy: expose what each role needs

Section: Block 2 / Security & privacy. **Presenter:** Simon + Matheus.

Campaign use and incident investigation require different views of customer information.

- **Minimize collection** and the information exposed in outputs.
- **Separate views:** campaigns and investigators have different purposes.
- **Protect logs** and define retention and access rules.
- **Preserve accountability** without making raw evidence broadly available.

Bank example: Campaign staff may need Ana's approved score and eligibility context; investigators may need restricted versioned evidence.

Teaching point: Privacy is an access-and-data-use design, not a label attached to a technique.

Additional speaking context:

The campaign needs enough information to apply its permitted action policy. An authorized investigator may need protected inputs, versions and action history. Those are different views with different retention needs. Features, model outputs, explanations and logs can remain sensitive even when direct identifiers are replaced.

Reading anchor: Simon's notes, §5–7 · Bass, Ch. 9–10 / §14.2.

Next: Privacy techniques have specific limits — Simon + Matheus.

45. Privacy techniques have specific limits

Section: Block 2 / Security & privacy. **Presenter:** Simon + Matheus.

Choose a mechanism for a defined objective rather than claiming privacy by technique name.

- **Specify the objective:** what information should be protected from whom?
- **Tokens** and synthetic data do not automatically establish anonymity.
- **Federation** changes data location, not every leakage risk.
- **DP** needs a defined protected unit and accounting across releases; no derivation here.

Bank example: Protecting one transaction differs from protecting all contributions of a bank customer.

Teaching point: Do not announce a guarantee without its assumptions and evidence.

Additional speaking context:

The campaign needs enough information to apply its permitted action policy. An authorized investigator may need protected inputs, versions and action history. Those are different views with different retention needs. Features, model outputs, explanations and logs can remain sensitive even when direct identifiers are replaced.

Reading anchor: Simon's notes, §5–7 · Bass, Ch. 9–10 / §14.2 · NIST SP 800-226.

Next: Use the quality checklist as a review tool — TBD.

46. Use the quality checklist as a review tool

Section: Block 2 / Quality concerns. **Presenter:** TBD.

Connect concerns to architectural controls, ownership, and observable evidence.

- **System and process concerns** both matter in ML-intensive systems.
- **Select relevant checks** rather than treating a short list as the entire checklist.
- **Map each concern** to a control, owner, and evidence.
- **Keep limits visible:** the separate corrigendum remains unread.

Bank example: Completeness validation addresses input quality; it is not by itself evidence of SS5 cyberattack defense.

Teaching point: A checklist supports structured review; it is not a certification claim.

Additional speaking context:

For SMN3, connect input-distribution monitoring to a dated cohort comparison and investigation record. For SA1, connect recovery to a rehearsed compatible restoration and verified outputs. For SS5, connect cyberattack defense to enforced source/release permissions and evidence that prohibited operations are rejected. A normal data-validation check alone does not demonstrate that security defenses are effective.

Reading anchor: Bucaioni, published Table 1, pp. 6–7 · Bass, Ch. 14 · Corrigendum contents not reviewed.

Next: Predictive quality and system performance — TBD.

47. Predictive quality and system performance

Section: Block 2 / Quality concerns. **Presenter:** TBD.

Choose an acceptable model and system from several quality objectives.

- **Accuracy** alone is an incomplete objective, especially with imbalanced outcomes.
- **Performance** includes the end-to-end deadline and resource use.
- **Complexity** must justify training and operating costs.
- **Acceptance** depends on the task and the consequences of errors.

Bank example: A heavier churn model is useful only if its quality benefit justifies cost and still meets the campaign's deadline.

Teaching point: Define acceptance criteria before selecting a more complex model.

Additional speaking context:

Precision asks how many flagged customers actually churned; recall asks how many actual churners were found. Calibration asks whether predicted risks agree with observed outcome rates in an appropriate evaluation. These measures answer different questions and need reliable mature labels. The batch deadline and operating cost are system measures, so they can be assessed separately from future churn quality. Do not invent numerical thresholds or claim an evaluated winner between models.

Reading anchor: Bucaioni, published Table 1, pp. 6–7 · Bass, Ch. 14 · Kalinowski, §13.4.

Next: Robustness: invalid, unfamiliar, or ordinary? — TBD.

48. Robustness: invalid, unfamiliar, or ordinary?

Section: Block 2 / Quality concerns. **Presenter:** TBD.

Different inputs need different responses; do not treat every unusual value as the same failure.

- **Contract validity** and model applicability are distinct.
- **OOD inputs** may be structurally valid but poorly represented in training.
- **Confidence** alone is not a guaranteed detector of unfamiliar inputs.
- **Fallbacks** must be validated for the intended situation.

Bank example: A missing activity value is invalid; a new customer population may be valid but outside the model's reliable coverage.

Teaching point: An architecture needs an explicit response when the model should not be trusted.

Additional speaking context:

Use the notes' invented mature cohort if needed: 12 of 20 flagged customers leave, giving 60% precision; 12 of 30 actual churners were flagged, giving 40% recall. Those answer different questions. Calibration asks whether probability estimates agree with observed frequencies in comparable groups; the notes' group near 0.70 with 0.40 observed churn illustrates overprediction for that group only.

Reading anchor: Bucaioni, published Table 1, pp. 6–7 · Bass, Ch. 14 · Simon's notes, §2–4.

Next: Explainability and auditability differ — TBD.

49. Explainability and auditability differ

Section: Block 2 / Quality concerns. **Presenter:** TBD.

Explaining feature influence and reconstructing a historical decision are complementary tasks.

- **Explanation** helps interpret model behavior in context.
- **Auditability** needs historical evidence and traceable versions.
- **Decision review** includes business policy and action, not only the model.
- **Limits:** SHAP and LIME do not prove causality or fairness.

Bank example: An engagement contribution may help explain Ana's score; it cannot prove why she would leave the bank.

Teaching point: Protect evidence and test whether authorized reviewers can actually retrieve it.

Additional speaking context:

The notes' fictional comparison improves overall recall from 40% with M1 to 50% with M2, while group A falls from 40% to 30%. The tiny group counts demonstrate the arithmetic, not a statistically reliable production verdict. If using the notes' 35% per-group gate, label it an invented teaching rule. Rejecting M2 does not certify that M1 is suitable forever.

Reading anchor: Bass, Ch. 14 · Simon's notes, §6–7.

Next: Fairness: prediction versus treatment — TBD.

50. Fairness: prediction versus treatment

Section: Block 2 / Quality concerns. **Presenter:** TBD.

Name what is being compared before selecting a metric or declaring an improvement.

- **Prediction fairness:** compare errors or calibration across relevant groups.
- **Treatment fairness:** inspect access to offers and policy effects.
- **Context:** select targets based on harms and requirements.
- **Trade-offs:** one metric does not settle every fairness concern.

Bank example: Equal churn prediction errors do not imply equal access to retention offers under campaign allocation rules.

Teaching point: Keep predictive quality and the consequences of intervention distinct.

Additional speaking context:

The notes' fictional comparison improves overall recall from 40% with M1 to 50% with M2, while group A falls from 40% to 30%. The tiny group counts demonstrate the arithmetic, not a statistically reliable production verdict. If using the notes' 35% per-group gate, label it an invented teaching rule. Rejecting M2 does not certify that M1 is suitable forever.

Reading anchor: Bucaioni, published Table 1, pp. 6–7 · Bass, Ch. 14 · Bass, Ch. 10 / §14.2.

Next: Airflow: make workflow dependencies visible — TBD.

51. Airflow: make workflow dependencies visible

Section: Block 2 / Practical tools. **Presenter:** TBD.

Orchestration supports task order, failure handling, and execution evidence.

- **Architecture role:** order dependent pipeline work.
- **Evidence:** task state, timing, and run context.
- **Action:** operators and source owners inspect failed dependencies.
- **Limit:** orchestration does not supply predictive or release approval evidence by itself.

Bank example: The required CRM validation must sit upstream of scoring and export so its failure blocks dependent work.

Teaching point: Connect each tool to a responsibility, evidence, and a response owner.

Additional speaking context:

Introduce the display as a prepared illustrative workflow, not a captured Airflow run. Required input validation is upstream of scoring and campaign export. When validation fails, the dependent work must not proceed. The record shows execution, timing and failure context so operators and source owners can locate the affected batch.

Reading anchor: Bass, Ch. 14 · Illustrative Airflow role.

Next: MLflow: connect experiments and artifacts — TBD.

52. MLflow: connect experiments and artifacts

Section: Block 2 / Practical tools. **Presenter:** TBD.

Model tracking supports reproducibility and review when evidence links are designed explicitly.

- **Architecture role:** connect training evidence and model artifacts.
- **Evidence:** runs, configuration, metrics, and artifact identity.
- **Action:** reviewers compare candidates and verify compatible releases.
- **Limit:** no single tracking tool provides every governance control.

Bank example: Link the approved M1 to its evidence while recording B017-R1's separate input and execution context.

Teaching point: Tool records support the release decision; they do not replace its policy.

Additional speaking context:

Show a model artifact linked to its training run, data reference and evaluation metrics. A complementary validation report describes required feature checks and the affected batch. The release record links this evidence to the approved compatible versions and recovery reference. Reviewers use these links to assess a release; source owners investigate invalid inputs.

Reading anchor: Bass, Ch. 14 · Illustrative MLflow role.

Next: Validation and drift reports support diagnosis — TBD.

53. Validation and drift reports support diagnosis

Section: Block 2 / Practical tools. **Presenter:** TBD.

A report is useful when its reference, feature meaning, cohort, and response are clear.

- **Validation:** test declared expectations before use.
- **Drift comparison:** inspect change against a named reference.

- **Tool choice:** reports can support these roles; keep policy outside the screenshot.
- **Action:** preserve evidence and assign a response owner.

Bank example: A missing CRM feature needs a contract-failure response; a changed valid population needs a different investigation.

Teaching point: Tools make evidence easier to collect; architecture defines what that evidence means.

Additional speaking context:

Show a model artifact linked to its training run, data reference and evaluation metrics. A complementary validation report describes required feature checks and the affected batch. The release record links this evidence to the approved compatible versions and recovery reference. Reviewers use these links to assess a release; source owners investigate invalid inputs.

Reading anchor: Simon's notes, §2–4 · Bass, Ch. 11 / §14.1.5 · Group tool-showcase topic.

Next: ML architecture connects five responsibilities — All presenters.

54. ML architecture connects five responsibilities

Section: Block 2 / Practical tools. **Presenter:** All presenters.

Return to the full system and the choices that make a prediction usable.

- **Data:** preserve meaning, validity, and temporal consistency.
- **Models:** evaluate uncertainty and manage compatible approved artifacts.
- **Operations:** observe, diagnose, release, and recover.
- **Decisions:** protect access and govern appropriate business action.

Bank example: Ana's score becomes usable through validated inputs, traceable execution, an approved release, and a separate campaign policy.

Teaching point: Day 2: foundation models and agents retain these control needs, while components and failure modes change.

Additional speaking context:

Ana's original illustrative score was 0.70. The CRM failure showed that usable predictions depend on compatible inputs, controlled change and appropriate action. We located an upstream fault, held B017, defined the boundaries it crossed, and explained how B017-R1 would be validated before use. We did not establish her eventual churn or the benefit of an offer.

Reading anchor: Bass, Ch. 14 · Group synthesis.

Next: Sources and adaptation boundaries — All presenters.

55. Sources and adaptation boundaries

Section: Block 2 / Practical tools. **Presenter:** All presenters.

The bibliography supplies concepts; the bank diagrams and incidents are explicit teaching applications.

- **Core readings:** architectural decisions, serving, maintenance, and quality concerns.
- **Supporting readings** extend monitoring, governance, and security.
- **Illustrations:** Ana, scores, dates, batches, and incidents are invented.
- **Source limit:** the separately catalogued checklist corrigendum remains unread.

Bank example: No new churn outcome, repaired score, retention benefit, or real bank tool execution is claimed.

Teaching point: The separate guide records reading anchors and adaptation limits.

Reading anchor: See source-review.md and presenter guide.

Source and adaptation notes

The inspected reading locations and limitations are recorded in SOURCES.md. Kalinowski Chapters 13–14 supply the assigned architecture and design material; Chapter 12 and §§16.3–16.5 are supporting reading, not silently relabeled assigned chapters. Bass Chapters 9–11 and 14 supply security, privacy, observability and the banking case. Nazir references are selected decision/discussion excerpts rather than a fresh complete paper read. Published Bucaioni Table 1 was spot-checked on PDF pages 6–7. Its separately catalogued corrigendum has not been read.

Selected checklist codes are part of a larger checklist. SMN3 concerns input monitoring; SA1 recovery; SS5 cyberattack defense; SS6 safety/fairness; SP1 privacy considerations. Ordinary input completeness checks alone do not establish cyber defense. A checklist prompt, SHAP/LIME attribution, federated learning, or a token does not by itself prove fairness, causality, compliance, or a privacy guarantee.

All architecture figures are teaching syntheses, not exact book-figure copies. Data-drift counts and concept-drift curves are schematic invented illustrations. Airflow and MLflow records are prepared explanations, not observed tool runs. No new full-text source checks or new experimental results are claimed by this visual reorganization. Mathematical detail remains in the separate personal notes.